

Forecasting Departure-Level Demand Seven Days Ahead for a Seasonal Mountain Railway

Efstratios Voulgaris

MSc Data Science & Information Technologies

September 29, 2026

Abstract

We forecast the final number of guests on each scheduled departure of a seasonal cogwheel railway seven days before travel (T-7), using only information available at that moment, and flag the departures that will run at least 90% full. At T-7 only about half of the final demand is on the book. We used both data enrichment techniques and excluded irrelevant information to improve the predictive power of our models. We split the data into training and validation folds keeping the date order. All of our experiments/evaluations are conducted on the last two folds because they resemble the test situation. On these folds, the strongest planning heuristic reaches a mean absolute error of 25.96 guests per departure, Ridge regression with selected interactions 24.98, and a tuned multi-layer perceptron averaged over five seeds 22.19. All models under-forecast a new season that follows the winter gap. For the fullness flag, a linear support vector machine reaches an average precision of 0.650, against a prevalence of 0.12–0.16. On the 2026 test season, which we evaluated once, the MLP reaches 21.74 against 27.43 for the heuristic, and the fullness classifier an average precision of 0.738.

Contents

1	Problem description	3
1.1	Business context	3
1.2	Forecasting task	3
1.3	Fullness classification task	3
1.4	What success means	3
2	Problem challenges	4
2.1	Leakage at the T-7 cutoff	4
2.2	Leakage through the capacity counters	4
2.3	Non-stationarity	5
2.4	A new season after a winter gap	5
2.5	Many empty departures and a heavy tail	5
2.6	A rare, shifting positive class	5
2.7	Correlated departures within a day	6
3	Dataset description	6
3.1	Source and grain	6

3.2	Seasons and population	6
3.3	Target	6
3.4	Demand streams	7
3.5	The T-7 snapshot	7
3.6	Booking curves	8
3.7	Features	8
4	Baselines	9
5	Methodology	9
5.1	Validation design	9
5.2	Metrics	10
5.3	Preprocessing	10
5.4	Linear models	10
5.5	Multi-layer perceptron	10
5.6	Booking-curve sequence models	12
5.7	Fullness classification	12
5.8	Test protocol	13
6	Results	13
6.1	Baselines and linear models	13
6.2	Multi-layer perceptron	14
6.3	Booking-curve sequence models	15
6.4	Fullness classification	16
6.5	Per-fold view and bias	17
6.6	Test season 2026	17
7	Limitations and next steps	19
A	Decision log	21

1 Problem description

1.1 Business context

The client operates a cogwheel mountain railway in Central Switzerland with connecting lake boats. It runs seasonally, roughly from May to November, with about 35 scheduled departures per operating day. Staff rosters and extra carriages are decided about one week before travel. At that point the reservation system shows only about half of the guests who will finally travel: 45% of the guests who travel had booked by then (Table 3). The rest are late individual bookings and walk-ups. Planning on the visible book alone under-provisions systematically. Planning on a long-run average ignores what the book already says about a specific day.

1.2 Forecasting task

For every scheduled departure, we predict the final number of guests seven days before travel (T-7), using only information that is known at T-7 [D1]. One row is one scheduled departure on one day, which we call a *route-day*. Route-day forecasts sum to the daily totals that operations plan with, so the finer grain loses nothing. A day-level model would have only about 650 rows. T-7 is the latest point at which operations can still act (roster staff, add a carriage). It is also the earliest point at which the book holds a useful share of the final demand.

1.3 Fullness classification task

The second output is a flag that says whether a departure will run at least 90% full. A departure is labelled *nearly full* when its final number of guests reaches 90% of the capacity it actually ran with. The flag is raised at the same T-7 decision point, from the same information. It tells operations where an extra carriage should go.

1.4 What success means

We judge a model on four criteria.

- It beats the planner’s heuristic. The reference is PickupBlend, a revenue-management heuristic that combines the current book with historical pickup (§4). A learned model is useful only if it beats PickupBlend on the validation folds that resemble the test situation and, finally, on the sealed 2026 season.
- Its error in guests is low at route and day level. The headline metric is the mean absolute error (MAE) in guests per route-day. We also report the error of daily totals (§5.2).
- Its bias is low. Forecasts that are systematically low leave guests behind, and a model can have a good MAE and still be biased.
- For the fullness flag, it catches 80% of the nearly-full departures with as few false alarms as possible.

In the rest of the report, “folds 4–5” are the two validation folds that resemble the test situation (§5.1). MAE is in guests per route-day unless stated otherwise. [Dn] points to entry *n* of the decision log in Appendix A.

2 Problem challenges

2.1 Leakage at the T-7 cutoff

Every input must be rebuilt as it stood at T-7. For bookings, this means a cutoff on the creation time (§3.5). Features built from past departures may use only departures that were complete by the cutoff. A departure seven days earlier falls on the cutoff day itself, so history lags start at 14 days [D6].

Bookings that are later cancelled. The database stores each booking’s final status but no cancellation timestamp. For a booking that was created before T-7 and later cancelled, we cannot tell whether it was still live at T-7 or already cancelled [D7]. Such bookings are 14.3% of the guests on the T-7 book. Without them, the book correlates slightly better with the target offline (0.6468 against 0.6354). However, the exclusion uses knowledge that does not exist at T-7. In production the forecaster sees these bookings on the book, so a model trained without them would receive a different quantity from the one it learned. We therefore include them. The remaining error is bounded and has a known direction: cancellations that happened before T-7 are counted as live, which slightly overstates the book.

2.2 Leakage through the capacity counters

The `DailyRoutes` table has, per departure and day, a total capacity and its split into group, individual and reserve seats. These columns look like properties of the train, but they are allocation counters that the booking system updates as bookings arrive, and the database holds only their final values. We removed the leak in three rounds [D11].

- (a) In the first round we looked at `group_capacity`. It equals the booked group seats in 95.5% of rows and correlates with the group component of the target at 0.9926, against 0.8815 for the legitimate T-7 group book. Deviations are rare in both directions (3.0% of rows below, 1.5% above), which confirms that the column is a counter. We excluded it.
- (b) The total, individual and reserve capacities stayed in the feature set. Total minus individual minus reserve gives the group capacity exactly, so in the second round we removed the total.
- (c) A later audit showed that the total is exactly 188 seats on about 80% of departures. Group capacity could therefore still be recovered as $188 - (\text{individual} + \text{reserve})$. The individual pool is itself a counter: it equals the booked individual seats in 95% of 2023 rows. With these columns a model predicts the group component with an MAE of 1.96 guests, against 5.44 without them, and the demand models gained a spurious ≈ 1.4 MAE.

In the third round we removed all per-day allocation columns. We replaced them with the planned seat split from the season timetable, which is fixed before the season starts (§3.7). There is a second reason to prefer the plan. The actual total capacity is overridden by hand on about 15% of departures, and the overrides follow demand (correlation +0.29), so even the total is partly an outcome. An operational counter that is updated in response to demand leaks the target, even when its name says “capacity”. Removing a column does not remove its information when it can be rebuilt from other columns or from a constant. A leakage check must therefore test whether the information can be recovered from the remaining inputs.

The fullness label is defined on the capacity the train actually ran with, because it describes the outcome. A classifier that divides a demand forecast by a capacity uses the planned capacity.

2.3 Non-stationarity

A product launch changed what the data measure. Time tickets and tickets with online payment went live on 2023-10-27. Before that date, much of the individual demand was not recorded the way it is now, and its level cannot be reconstructed. Year-on-year growth over the same calendar days shows the break (Table 1). We compare 5 June to 9 November, the days that every development season covers. Individual demand grew by 60% into 2024 and by 21% the year after, while group demand was nearly flat. The 2023→2024 jump in individual demand therefore mostly reflects the change in how demand is recorded. A season index would read it as growth and extrapolate it [D13].

Table 1: Year-on-year growth of final demand over the same calendar days (5 June to 9 November).

Transition	Total	Individual	Group
2023 → 2024	+32%	+60%	+7%
2024 → 2025	+11%	+21%	−3%

Real growth. Demand also grows between seasons. Because of the launch, the development data contain only one season-to-season transition that the launch does not affect (2024→2025). A growth rate can be learned from this transition alone.

Renumbered trains. All train numbers changed between 2024 and 2025: none of the 34 carries over. We therefore identify a departure by its timetable slot (departure station and time). Per-slot mean demand correlates at 0.93–0.99 between seasons [D9].

2.4 A new season after a winter gap

In the test situation the model forecasts the level of a season it has never seen. It is trained on earlier seasons, which the winter closure separates from the test season. Inputs whose value at prediction time lies outside the training range, such as a season index, are a particular hazard for neural networks. A network has no rule for extending beyond the range it was trained on (§5.5).

2.5 Many empty departures and a heavy tail

About 16.3% of route-days carry no guests at all, while a single group booking can bring 50 guests onto one departure. Zero targets make per-row percentage errors undefined. The heavy tail makes squared-error training sensitive to a few large groups. Both facts shaped our choice of metrics and loss (§5.2, §5.5).

2.6 A rare, shifting positive class

Nearly-full departures are 8.4% of the development set. Their share changes a lot between seasons: 0.4% of 2023 departures, 10.3% of 2024 and 14.1% of 2025. Accuracy is useless here, since always predicting “not full” scores above 90%. A decision threshold tuned where positives are rare may also fail to transfer to a period where they are common (§5.7).

2.7 Correlated departures within a day

The ~ 35 departures of one day share weather, holidays, events and crowding. If a day were split across training and validation, a model could learn that day’s conditions from its neighbours and the score would be inflated. Validation therefore always keeps whole days together (§5.1). For the same reason, the effective sample size for day-level effects, such as a specific holiday, is the number of days.

3 Dataset description

3.1 Source and grain

The data come from the operator’s reservation database (PostgreSQL). We use four core tables. **DailyRoutes** has one row per scheduled departure per day and holds the stored seat counters. **BookingTransportations** links each row of **Bookings** to its outbound and return legs and therefore to departures. **TimetableRoutes** is the season timetable with the planned seat configuration of each train. One modelling row is one **DailyRoutes** row, a route-day.

3.2 Seasons and population

We use the seasons 2023–2026 [D2]. We excluded 2021 and 2022, where individual seats are recorded as exactly zero and the median capacity was 80 seats against 188 from 2023 onwards. These two seasons describe a different operation. We removed special routes (charters, one-offs) and disabled (cancelled) departures (Table 2). We kept departures with zero demand. About 98% of the route-days with no attributable booking have a stored target of exactly 0, so these are real empty trains.

Table 2: Population funnel, seasons 2023–2026.

Step	Route-days	Days
All scheduled route-days	25,135	
minus special routes	−536	
minus disabled routes	−2,041	
Modelling population	22,558	649
development set, 2023-06-05 to 2025-11-16	18,002	515
sealed test set, 2026-05-11 to 2026-09-24	4,556	134

The 2026 season is the test set [D4]. We scored it exactly once, after all modelling decisions were fixed (§6.6). Its window is 2026-05-11 to 2026-09-24, and we fixed the end date in the configuration so that later data pulls cannot change it. We saw aggregates of 2026 during the exploratory analysis and in one descriptive growth table, but never used them for a modelling decision.

3.3 Target

The target is the stored **DailyRoutes** counter of total guests, group plus individual [D3]. It is the number the business reports. It agrees with the total rebuilt from bookings at horizon 0, with cancelled bookings removed: the correlation is 0.9999, 99.6% of departures lie within ± 5 guests, and the mean difference is +0.11. The target takes values from 0 to 194 guests.

3.4 Demand streams

We split bookings into three behavioural streams using the booking-type field [D8]. The share of each stream that is visible at T-7 differs strongly (Table 3). Group bookings are made early, with a median lead time of 27 days. On-the-spot demand is almost invisible a week ahead and can only be inferred from what a departure usually carries.

Table 3: Demand streams on the development set: share of travelling guests and the part of each stream already booked at T-7.

Stream	Booking types	Share	Visible at T-7
group	group bookings	43.9%	86.1%
individual_advance	with payment, via API	28.7%	23.0%
on_spot	time ticket, via partner, legacy nulls	27.4%	1.6%
all streams		100%	44.8%

The booking-type field is filled only from late October 2023. The earlier “legacy null” rows behave like walk-ups (median lead time 0 days), so we assign them to `on_spot`. We do the same for a handful of rows with an undocumented type.

3.5 The T-7 snapshot

A booking leg counts toward the T-7 snapshot of a departure only if it was created on or before travel date -7 [D6]. We verified the rule in two independent ways. A second implementation (calendar dates instead of lead times) agrees with the first, and the same code at horizon 0 reproduces the stored target (correlation 0.9999 above). Table 4 splits the guests of the development set by creation time and final status. Part B is the later-cancelled share discussed in §2.1. Measured against the guests who travel, the T-7 book holds 52.3% with B and 44.8% without it. The second figure is the one in Table 3.

Table 4: Guests by booking creation time and final status (development set).

Part	Definition	Guests
A	created $\leq$ T-7, travelled	376,754
B	created $\leq$ T-7, later cancelled	62,927
C	created after T-7 (pickup)	463,576

The snapshot has 13 columns: guests and bookings per stream and in total, guests booked in the last 3 and 7 days before the cutoff, mean party size, days since the last booking, and an empty-book flag.

The fill value for an empty book. “Days since the last booking” is undefined for an empty book (about a third of route-days). We first filled it with 60 [D10]. Live values reach 338 days (90th percentile 126), so 17% of real books looked staler than an empty one. Yet empty books end with 10.5 guests on average, and books whose last booking is really 60 days old end with 69.6. The fill value is now 999, above every observed value, with an explicit `book_is_empty` flag. The feature itself is informative: pickup after T-7 falls from +47.6 guests when the last booking arrived on the cutoff day to +19.8 when it is 31 or more days old.

3.6 Booking curves

For the sequence models, each departure also has its full booking curve: the cumulative number of guests on the book on each day from T-60 to T-7, per stream. This gives a tensor of shape $(N, 54, 3)$. The same cutoff and cancellation rules apply at every step, so the last step equals the T-7 snapshot. A 60-day window covers 94.7% of booking legs. Bookings made earlier are kept and enter the T-60 value as an opening balance. Values are stored in guests; scaling is fitted per training fold.

3.7 Features

Every feature belongs to exactly one group that can be switched on or off, and the group name is the column prefix. An ablation is therefore a column filter. The matrix has 47 columns (Table 5).

Table 5: Feature groups (number of columns in parentheses).

Group	Content
book (13)	T-7 snapshot (§3.5)
calendar (6)	weekday, weekend flag, month, day of year with sine/cosine encoding
route (3)	timetable slot (station + departure time), departure station, departure hour
timetable (4)	planned group, individual, reserve and total seats from the season timetable
boat (2)	boat-connected departure, boat time
trend (3)	season index, days into season, season progress
regime (1)	step flag for the 2023-10-27 product launch
holiday (10)	per-canton flags, number of cantons, holiday identity, bridge day, distance to the nearest holiday
history (5)	same departure, same weekday, 14/21/28 days earlier, their mean, missing flag
weather (0)	deferred (§7)

Planned capacity. The timetable group replaces the per-day capacity counters of §2.2 [D12]. Each season’s timetable fixes, per train, how its 188 seats are split between the group, individual and reserve pools, for example 80/98/10 or 140/38/10. We join it on train number, departure station and a travel day within the validity dates of the timetable entry. Every departure matches exactly one entry. The plan exists long before the season (the next season’s timetable is already in the database), so it is known at T-7. It tells the model which departures the timetable reserves for groups. The slot cannot express this, because a slot’s allocation changes between seasons.

Regime flag. We model the launch of §2.3 as a step dummy and keep all rows [D13]. The 2023 rows still carry valid group demand, the slot and weekday pattern, and extra holiday observations. The flag absorbs the change of measurement, so the season index measures growth. Its value depends only on the travel date, and both of its values occur in every training fold.

Holidays. We measured holiday effects against neighbouring days with the same weekday. They are large and differ in sign [D14]. Whit Monday raises demand by +111% to +211% and Assumption Day by +64% to +117%, whereas Swiss National Day lowers it by −54% to −59% (three measurements each). The number of departures is the same on these days, so the effect is on demand. Because the sign reverses, we encode which holiday it is instead of a single flag.

We use the Central Swiss cantonal calendar (six cantons). Across its 22 holidays the mean lift is +38%, whereas federal-only and foreign calendars carry no independent signal.

4 Baselines

Before any learning, we built five forecasts without machine learning. They show whether machine learning is needed at all and set the bar a learned model must clear [D17]. Each is fitted on the training fold only.

- GlobalMean predicts the training mean. It is the floor.
- SeasonalNaive(14) predicts the same slot 14 days earlier. The lag is 14 days because a departure seven days back is not yet complete at the T-7 cutoff.
- SlotMonthDowMean predicts the mean for this slot, month and weekday. For combinations not seen in training it falls back to coarser keys. This is roughly what an experienced planner would do with a spreadsheet.
- PickupRatio divides the T-7 book by the share of final demand that is usually visible at T-7 for that slot and month. The share is estimated as a ratio of sums. Empty or near-empty books fall back to SlotMonthDowMean.
- PickupBlend averages PickupRatio and SlotMonthDowMean. The two fail in different ways: the book says nothing when it is empty, and the historical mean cannot see an unusually busy week.

PickupBlend is the strongest of the five, with an MAE of 25.96 on folds 4–5 against 47.51 for GlobalMean (Table 8). It is the bar to beat. PickupRatio shows why we need more than one metric. Its route-level errors are large and go both ways (RMSE 53.2, the worst of the informative baselines). When summed to the day they largely cancel, so its day-level error (34.2%) is close to PickupBlend’s (33.7%).

5 Methodology

5.1 Validation design

Plain k -fold cross-validation does not fit this problem, for two reasons [D5]. It trains on days that come after the days it validates, and it splits the correlated departures of one day across folds. We used expanding-window cross-validation with $k = 5$ on the development set. Folds are cut on whole days, and training always strictly precedes validation. We split each season into two halves, so that every validation window lies inside one season (Table 6). If windows straddled the winter closure, fold scores would differ by their seasonal mix more than by model.

Table 6: Expanding-window folds (training = all development days before the window).

Fold	Training data	Train rows	Validation window	Role
1	2023 to 2023-08-20	2,926	2023-08-21 to 2023-11-09	reported
2	2023	5,804	2024-05-17 to 2024-08-13	reported; one training season
3	2023, early 2024	8,830	2024-08-14 to 2024-11-17	reported
4	2023–2024	11,702	2025-05-12 to 2025-08-12	ranking; new season
5	2023–2024, early 2025	14,922	2025-08-13 to 2025-11-16	ranking

Ranking folds. We rank models on folds 4 and 5 only. They are the only folds with at least two training seasons, as the final model has. Fold 2 trains on 2023 alone and must bridge the

product-launch break (Table 1). Its errors are dominated by that single jump, and it can reverse conclusions that hold on every other fold. Fold 4 is the closest preview of the test, because it validates the first months of a season that follows a winter gap. Fold 5 continues a season that is already partly in training. We always report per-fold results next to the mean, because the two folds often disagree.

5.2 Metrics

The headline metric is the mean absolute error (MAE) in guests per route-day [D16]. It is in the unit operations think in, and a few large groups do not dominate it. We also report the root mean squared error (RMSE), the coefficient of determination R^2 , and the bias

$$\text{bias} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i).$$

A model that is systematically low can have the same MAE as an unbiased one, so we report the bias separately. We also sum route-day forecasts to days and report the day-level MAE relative to the mean daily total (Day-MAE%). We do not use a per-route-day percentage error (MAPE), because it is undefined for the 16% of route-days with zero demand. R^2 compares a model with predicting the validation mean, so it moves with the variance of a fold and cannot be compared across folds. For neural models trained with several seeds, “(k seeds)” denotes the mean of the per-seed metrics. A seed ensemble averages the predictions, which makes it a different model.

5.3 Preprocessing

We fit all preprocessing inside each training fold, as one pipeline shared by the linear models, the networks and the classifiers [D18]. No statistic of the validation window reaches a model, and all models see the same preprocessing. Numeric inputs are median-imputed and standardised. The history lags are missing for about 9% of rows (early season), and the `hist_missing` flag marks these rows. Nominal codes (slot, departure station, weekday, month, holiday identity) are one-hot encoded. A single slope across slot numbers or holiday codes would be meaningless, and the holiday effect even reverses sign between levels. The 47 raw features become about 100 model inputs.

5.4 Linear models

We fitted ordinary least squares (OLS) and Ridge regression on the preprocessed features and chose the Ridge penalty α on folds 4–5 [D19]. The design matrix is ill-conditioned by construction: it has many correlated dummies, and the history mean is an exact linear combination of the three lags. We tested two extensions. The first adds polynomial terms on ten numeric drivers (group, individual and total book, staleness, history mean, departure hour, boat flag, day-of-year sine and cosine, season index). We used either all 45 pairwise products (interactions only) or the full degree-2 expansion, which adds the ten squares. The second fits a log target, $\log(1 + y)$, and transforms the prediction back.

The linear models keep the season index, because a linear model can safely extend the growth into a new season as a straight line.

5.5 Multi-layer perceptron

Design. The network stacks blocks of Linear $\rightarrow$ normalisation $\rightarrow$ ReLU $\rightarrow$ Dropout. The normalisation is batch normalisation, layer normalisation or none. Weights use He initialisation,

and the optimiser is AdamW. The loss is L1, Huber or MSE. The target is standardised during training and transformed back afterwards. With the L1 loss the network estimates the conditional median, which suits the MAE headline. A negative mean bias under L1 therefore does not hurt the MAE, but it matters for daily totals.

Early stopping on the training fold. If we stopped training when the validation score peaks, we would tune on the evaluation data. We therefore split each training fold again in time order. The last 15% of its operating days decide when to stop, and the best weights are restored [D20]. The validation fold is never used for this.

The season index is withheld from networks. In fold 4 the season index is 0 or 1 in training and 2 at validation. For the test it is 3. A linear model extends its slope to the unseen value. A network has no such rule and maps the value to an arbitrary output. Networks therefore never receive inputs whose value at prediction time lies outside the training range [D21]. The regime flag is safe, since both of its values occur in every training fold. We also tried training the network on the residuals of Ridge, which does extrapolate. It did not help, because Ridge’s in-sample residuals contain no level error for the network to learn.

Tuning protocol. A single training run changes with the random seed by more than most differences between settings, so comparisons of single runs are unreliable. We scored every candidate configuration by its mean MAE over 3 seeds on folds 4–5, with the standard error (SE) of that mean. The *one-standard-error rule* treats candidates within one SE of the best as ties, and the simpler candidate wins a tie [D22]. We ran two search methods under these rules on the same data and folds (Table 7).

- Random search samples all factors jointly, so it explores interactions such as batch size $\times$ learning rate instead of assuming they are absent. Sampled configurations have no natural order, so a tie goes to the configuration with the fewest trainable parameters.
- Greedy sequential search (coordinate descent) tunes one factor at a time in a fixed order and keeps each winner. Candidates are listed in an order of preference fixed in advance: L1 first, because it matches the metric; then fewest components, smallest network, more regularisation, larger batch and the default learning rate. The first-listed candidate within one SE of the best wins. The result depends on the path, because early factors are decided while later ones are still at their default values.

We also considered quasi-random search, Bayesian optimisation and Hyperband, and kept random search as the reference method. All runs are seeded.

Table 7: MLP search spaces. Greedy candidates are listed in order of preference; the default configuration is the greedy start and trial 1 of the random search.

Factor	Random search (joint space)	Greedy (order of preference)	Default
Loss	L1, Huber, MSE	L1, Huber, MSE	Huber
Normalisation	none, layer, batch	none, layer, batch	batch
Hidden layers	[64], [128], [128,64], [256,128], [256,128,64]	[64], [128,64], [256,128,64]	[128,64]
Dropout	uniform [0, 0.5]	0.3, 0.1, 0.0	0.1
Weight decay	log-uniform [10^{-6} , 10^{-1}]	10^{-2} , 10^{-4} , 0	10^{-4}
Batch size	256, 1024, 4096	4096, 1024, 256	256
Learning rate	log-uniform [10^{-4} , 10^{-2}]	10^{-3} , $3 \cdot 10^{-4}$, $3 \cdot 10^{-3}$	10^{-3}
Tie-break (1 SE)	fewest trainable parameters	first-listed candidate	
Budget	21 trials	21 configurations	

Seed ensembling. We also evaluated the chosen configuration as an ensemble that averages the predictions of 5 networks that differ only in their seed [D23]. Averaging cancels part of the seed variance, so the reported number depends less on one lucky or unlucky initialisation.

5.6 Booking-curve sequence models

With the sequence models we ask how much the booking curve (§3.6) alone tells us about final demand, and which encoder reads it best [D24]. The models see only booking curves and no tabular features. The input is the departure’s own curve: 54 days (T-60 to T-7) by 3 streams (group, individual_advance, on_spot), in cumulative guests. We transformed it with $\log(1+x)$, since a single group can add 50 guests, and standardised each channel with statistics of the fitting rows. We compared four encoders, each followed by one linear output:

- 1D-CNN: convolutions along time (kernel 5, about a working week), then mean and max pooling over time;
- LSTM and GRU: the last hidden state, at T-7, summarises the curve;
- Transformer encoder: self-attention over the 54 days with a learned positional embedding, because attention alone ignores order, followed by mean pooling.

All four have 64 hidden units, one layer and dropout 0.1. An earlier search over size and depth changed the MAE by at most about 0.3, so our ranking is a ranking at equal size. Training follows the MLP: L1 loss, AdamW with learning rate 10^{-3} , batch 256, weight decay 10^{-4} , and early stopping on the last 15% of the training fold’s days. We trained 5 seeds per encoder on folds 4–5.

Reference curve. The own curve cannot see walk-ups. They make up about 27% of guests and are almost never booked a week ahead (Table 3), so the plain encoders under-forecast by about 12 guests per departure (§6.3). In a second experiment we gave the two best encoders a reference curve as 4 extra channels, which makes the input 54 by 7. The reference is the same timetable slot’s departures 14, 21 and 28 days earlier, on the same weekday, averaged over those that exist. The departure 7 days earlier runs on the cutoff day, so 14 days is the nearest same-weekday departure that is complete before the cutoff. The reference channels are aligned on the same calendar days as the own curve. Before its departure day, a reference channel shows that departure’s book on that calendar day. From its departure day on, it holds the guests who actually travelled, so it carries the late bookings and walk-ups that the own curve cannot see at T-7. The fourth extra channel is the number of references found (0–3), constant over the 54 days, so that “no reference” differs from “a reference with no guests”. All three references exist for 79% of departures and none for 9%; 95% of the latter fall in the first 14 days of a season, after the winter gap. The reference uses only bookings created on or before the forecast’s cutoff, and this also holds for the few bookings recorded after their travel date. The final count of a reference carries information similar to the history lags of the tabular features, delivered as part of a curve.

5.7 Fullness classification

The label is final guests $\geq 90\%$ of the actual capacity, and the inputs are the same 47 features [D25]. We compared the perceptron, a linear support vector machine (SVM), logistic regression, Gaussian naive Bayes, k -nearest neighbours, an RBF-kernel SVM and a decision tree. All of them use the shared preprocessing and, where applicable, balanced class weights. We also trained a neural classifier: the MLP with a logit output, trained on cross-entropy with positives up-weighted by the ratio of negatives to positives. Finally, we built two regression-derived flags that divide a demand forecast (the MLP’s or PickupBlend’s) by the planned capacity. These

flags test whether a dedicated classifier beats a threshold on a good demand forecast.

Ranking. We rank models by average precision (AP), the area under the precision–recall curve. AP needs no threshold and stays informative when positives are rare. A classifier with no skill scores the prevalence (0.164 on fold 4, 0.117 on fold 5). We report ROC-AUC next to it.

Operating point. We set the threshold for a target recall of 80%: we catch 80% of the nearly-full departures and count how many false alarms that costs. Recall is computed among positives only, so a threshold chosen for a target recall does not depend on how common positives are. An F1-optimal threshold does depend on it and would not transfer from quiet to busy weeks. Everything a classifier learns comes from the training fold: the weights, its one hyperparameter (from a small grid, chosen by AP) and the threshold. We fit on the first 85% of the fold’s days and choose on the last 15%. The validation fold is only scored.

5.8 Test protocol

We fitted every test model once on the whole development set (18,002 route-days, 2023-06-05 to 2025-11-16) and scored it once on the 2026 window (4,556 route-days, 134 days) [D4]. We fixed the end date in the configuration before opening the test, so later data pulls cannot change the test set. Early stopping and classification thresholds use the last 15% of the development days.

We declared the candidates, the expected result (§6.5) and the error analysis before the run. The primary regression model is the final MLP (greedy pick) as a 5-seed ensemble. Ridge with interactions and the five baselines are references. We took to the test only models that were competitive in development, so no sequence model went to the test (§6.3). The primary classifier is the linear SVM with its threshold set for 80% recall; logistic regression and the two forecast / planned-capacity flags are secondary. The 95% confidence intervals come from a day-block bootstrap: 2,000 resamples of the 134 test days, in which the departures of one day stay together.

6 Results

Sections 6.1–6.5 report folds 4–5 of the development set. Table 8 ranks every regression model, and the subsections discuss each family. Section 6.6 reports the 2026 test season.

6.1 Baselines and linear models

Regularisation helps only a little. OLS reaches 26.41. Ridge gives 26.43, 26.60, 26.58, 25.99 and 26.20 for $\alpha = 0.1, 1, 10, 100, 1000$. The best plain Ridge ($\alpha=100$, 25.99) only ties PickupBlend on MAE, although it is better on RMSE, R^2 , bias and day-level error. Its largest standardised coefficients belong to the timetable-slot dummies and the holiday identity, followed by the group and total book at T-7.

Interactions beat full polynomials. Adding the 45 pairwise products gives 24.98 with almost no bias (−0.19). This is the first model that beats PickupBlend on every metric [D19]. The full degree-2 expansion is worse (25.96), and most of the gap is on fold 4 (27.14 against 25.89; fold 5: 24.77 against 24.08). The cause is the square of the season index. In the fold-4 training data the index takes only the values 0 and 1, where $\text{index}^2 = \text{index}$, so Ridge splits the season effect between two identical columns. At validation the index is 2 and its square is 4, so

Table 8: Regression models on folds 4–5, ranked by MAE (guests per route-day). MAE s.d. is across the two folds (and seeds). Day-MAE% is the MAE of daily totals divided by the mean daily total.

Model	MAE	MAE s.d.	RMSE	R^2	Bias	Day-MAE%
MLP, greedy pick, 5-seed ensemble	22.19	2.76	36.93	0.615	−3.34	29.3
MLP, greedy pick (3 seeds)	22.29	2.14	37.25	0.608	−3.13	29.3
MLP, random-search pick, 5-seed ensemble	22.75	3.10	36.91	0.616	−4.10	29.9
MLP, random-search pick (3 seeds)	22.94	2.41	37.17	0.610	−3.86	30.0
Ridge + interactions ($\alpha=1000$)	24.98	1.28	37.45	0.604	−0.19	31.0
Ridge + full degree 2 ($\alpha=5000$)	25.96	1.67	37.99	0.592	+1.19	31.1
PickupBlend (bar to beat)	25.96	2.29	40.36	0.540	−3.54	33.7
Ridge ($\alpha=100$)	25.99	0.48	37.66	0.599	+0.40	32.5
SlotMonthDowMean	30.40	3.70	44.24	0.449	−9.83	38.0
Ridge + $\log(1 + y)$ target	30.43	5.70	60.02	−0.014	−7.94	34.8
PickupRatio	31.21	1.17	53.18	0.198	+2.75	34.2
SeasonalNaive(14)	34.03	0.53	48.22	0.340	−5.64	44.2
GlobalMean	47.51	4.01	60.80	−0.041	−9.11	42.8

the squared half of the effect overshoots. The interactions-only expansion does not contain that term.

Log target. Fitting $\log(1 + y)$ and transforming back gives 30.43, with a bias of -7.94 and a negative R^2 . The back-transformed prediction estimates a geometric mean instead of an arithmetic mean. The geometric mean is lower (Jensen’s inequality), and the error grows on the large departures. We keep the target on the original scale.

Planned capacity. Adding the planned seat split (§3.7) moved Ridge with interactions from 27.67 to 24.98, and its bias from $+7.8$ to -0.2 [D12]. The plan gives the model, without leakage, the information that the capacity counters leaked: which departures the timetable sets aside for groups.

6.2 Multi-layer perceptron

Search results. The default configuration scores 23.61 ± 0.08 . The greedy search (Table 9) chose L1 loss, layer normalisation, [128,64], dropout 0.3, weight decay 10^{-4} , batch 4096 and learning rate $3 \cdot 10^{-3}$, at 22.29 ± 0.08 . The one-SE rule decided three steps. [256,128,64] had a slightly lower mean than [128,64] but was within one SE, so the smaller network was kept. In the same way, weight decay 10^{-4} was kept over 0 and batch 4096 over 1024. The best trial of the random search (trial 6) was [256,128], dropout 0.48, batch normalisation, L1, learning rate $3.6 \cdot 10^{-3}$, weight decay $2.9 \cdot 10^{-2}$ and batch 1024, at 22.87 ± 0.09 with 60,161 parameters. The one-SE rule chose trial 13, which is within one SE and has 21,889 parameters: [128,64], dropout 0.28, batch normalisation, L1, learning rate $1.7 \cdot 10^{-4}$, weight decay $5.0 \cdot 10^{-6}$ and batch 1024, at 22.94 ± 0.07 . Both searches agree that the loss matters most. The three best random trials all use L1. The four L1 trials scored 22.87–23.58, the seven Huber trials 23.43–24.23 and the ten MSE trials 24.36–26.48. The greedy step shows the same order. Both searches also prefer high dropout: 0.28–0.48 in the three best random trials and 0.3 in the greedy pick. The greedy pick is the final MLP [D23].

Table 9: Greedy sequential search (MAE $\pm$ SE over 3 seeds, folds 4–5). The chosen candidate (bold) is kept for all later steps; ties within one SE go to the first-listed candidate.

Step	Candidates, in order of preference	Chosen
1 Loss	L1 22.89$\pm$0.19 ; Huber 23.61 $\pm$ 0.08; MSE 24.49 $\pm$ 0.14	L1
2 Normalisation	none 23.12 $\pm$ 0.07; layer 22.57$\pm$0.26 ; batch 22.89 $\pm$ 0.19	layer
3 Hidden layers	[64] 22.90 $\pm$ 0.21; [128,64] 22.57$\pm$0.26 ; [256,128,64] 22.51 $\pm$ 0.11	[128,64] (tie)
4 Dropout	0.3 22.44$\pm$0.08 ; 0.1 22.57 $\pm$ 0.26; 0.0 22.94 $\pm$ 0.10	0.3
5 Weight decay	10 ⁻² 22.59 $\pm$ 0.11; 10⁻⁴ 22.44$\pm$0.08 ; 0 22.43 $\pm$ 0.15	10 ⁻⁴ (tie)
6 Batch size	4096 22.47$\pm$0.05 ; 1024 22.41 $\pm$ 0.12; 256 22.44 $\pm$ 0.08	4096 (tie)
7 Learning rate	10 ⁻³ 22.47 $\pm$ 0.05; 3 $\cdot$ 10 ⁻⁴ 22.46 $\pm$ 0.05; 3 $\cdot$ 10⁻³ 22.29$\pm$0.08	3 $\cdot$ 10 ⁻³

Equal search budgets. Both searches had 21 configurations, each scored over 3 seeds on 2 folds [D22]. The greedy pick (22.29 $\pm$ 0.08) beats the random-search pick (22.94 $\pm$ 0.07) by about 0.6, and it also beats the best random trial (22.87). At equal budget, greedy search found the better configuration. The random sample is fixed by a seed, so this comparison rests on one sample of 21 configurations.

Withholding the season index. Table 10 confirms the rule of §5.5. When the networks get the season index, both picks get worse by 0.5–1.0 MAE. The pattern matches arbitrary extrapolation: on fold 5 the bias flips from slightly negative to positive, and on fold 4 the error grows.

Table 10: Season index withheld (the rule) or given, 3 seeds each.

Configuration	Season index	MAE	Fold 4		Fold 5	
		folds 4–5	MAE	Bias	MAE	Bias
Greedy pick	withheld	22.29	24.23	−5.44	20.35	−0.82
Greedy pick	given	23.28	24.96	−7.99	21.60	+2.91
Random-search pick	withheld	22.94	25.12	−6.17	20.75	−1.55
Random-search pick	given	23.64	25.40	−5.94	21.88	+3.00

Ensembling and comparison with the other models. The 5-seed ensemble improves the greedy pick from 22.29 to 22.19 and the random pick from 22.94 to 22.75. We expected small gains, because both configurations are already strongly regularised. The final MLP beats PickupBlend by 3.8 MAE and Ridge with interactions by 2.8. On these folds its daily totals are the most accurate (Day-MAE 29.3% against 33.7% for PickupBlend); on the test season Ridge and PickupBlend give better daily totals (§6.6). Its weakness is bias: it under-forecasts by 3.3 guests on average, against −0.2 for Ridge (§6.5). Tuning gained 1.3 MAE over the default configuration. All tuned scores are optimistic, because each is the best of many noisy estimates on the same folds. Only the test season gives an unbiased estimate.

6.3 Booking-curve sequence models

Own curve. The four encoders are practically tied (Table 11). Their MAE spans 0.14, and the Transformer and the GRU are identical to two decimals (25.07). All four under-forecast by about 12 guests per departure, because walk-ups do not appear in the curve. From the curve alone they come close to Ridge with interactions (24.98), are ahead of PickupBlend (25.96) and lie well behind the final MLP (22.19).

Table 11: Booking-curve models on folds 4–5 (5 seeds each; MAE $\pm$ SE of the per-seed means). Every encoder has 64 hidden units, one layer and dropout 0.1. The last three rows use all tabular features and are given for comparison.

Model	MAE						MAE		Bias	
	folds 4–5	RMSE	R^2	Bias	Day-MAE%	Fold 4	Fold 5	Fold 4	Fold 5	
<i>Own curve</i>										
Transformer	25.07±0.06	45.21	0.424	−12.15	33.6	27.30	22.84	−14.57	−9.74	
GRU	25.07±0.07	44.85	0.433	−11.96	33.5	27.43	22.71	−14.62	−9.29	
1D-CNN	25.14±0.03	44.63	0.438	−12.59	34.2	27.40	22.87	−14.78	−10.40	
LSTM	25.21±0.03	45.30	0.422	−12.64	34.1	27.63	22.79	−15.41	−9.87	
<i>Own curve + reference curve</i>										
Transformer	24.16±0.13	40.81	0.529	−3.24	33.2	25.01	23.31	−10.13	+3.65	
GRU	24.21±0.14	41.02	0.524	−3.37	33.6	24.82	23.59	−10.62	+3.88	
<i>Tabular features</i>										
MLP, final (5-seed ens.)	22.19	36.93	0.615	−3.34	29.3	24.14	20.24	−6.04	−0.65	
Ridge + interactions	24.98	37.45	0.604	−0.19	31.0	25.89	24.08	−3.77	+3.39	
PickupBlend	25.96	40.36	0.540	−3.54	33.7	27.58	24.34	−7.89	+0.81	

Reference curve. We added the reference curve to the two best encoders, the Transformer and the GRU. It improves the MAE by about 0.9 and removes most of the mean bias, from −12 to −3.3. Both models now beat Ridge with interactions and PickupBlend, but they remain about 2 guests behind the final MLP. The two encoders stay tied: they differ by 0.04, with an SE of 0.13. On fold 4, the new season, the gap to the MLP narrows (24.8–25.0 against 24.14). On fold 5 they are worse than the plain encoders (23.3–23.6 against 22.7–22.8) and over-forecast by about 3.7 guests.

Booking dynamics and the recent history of the same slot recover a good part of the signal. The calendar, the holidays and the timetable plan, which the curve models do not see, are worth about 2 guests per departure. With the reference curve the sequence models rank second among our model families, behind the MLP. We took only models that were competitive in development to the test, so no sequence model went to the test season [D24].

6.4 Fullness classification

Table 12 ranks the classifiers by average precision. The two neural rows use the final MLP configuration.

The simple linear classifiers do best: the linear SVM and logistic regression lead on AP and on false alarms. Summed over both folds, the linear SVM catches 621 of 888 nearly-full departures (267 missed) at the cost of 461 false alarms among 5,412 other departures. This is about 1.8 flagged departures per catch. A dedicated classifier beats a threshold on the forecast: AP 0.650 against 0.606 for the MLP forecast divided by planned capacity. The flag derived from PickupBlend is clearly weaker (0.515).

Naive Bayes fails because its independence assumption does not hold. The one-hot dummies of one variable are strongly dependent, and so are the stream, total and history features, so the model counts the same evidence many times. The threshold transfers only approximately. The achieved recall lies between 0.70 and 0.92 against the 80% target, because the threshold is chosen on the last weeks of the training fold and applied to a different part of the season.

Table 12: Nearly-full classification on folds 4–5, ranked by average precision (AP). Precision, recall and F1 are at the threshold chosen for 80% recall on the training fold. “Alarms per catch” is flagged departures per true positive. A no-skill classifier has AP equal to the prevalence (0.164 and 0.117).

Model	AP	ROC-AUC	Precision	Recall	F1	Alarms/catch
Linear SVM	0.650	0.917	0.573	0.711	0.630	1.76
Logistic regression	0.644	0.918	0.559	0.706	0.619	1.80
Neural classifier (3 seeds)	0.609	0.915	0.515	0.787	0.622	1.94
MLP forecast / planned cap. (3 seeds)	0.606	0.898	0.442	0.792	0.533	2.54
k -NN	0.592	0.890	0.463	0.754	0.572	2.17
Perceptron	0.559	0.877	0.378	0.800	0.507	2.72
Kernel SVM (RBF)	0.544	0.881	0.422	0.813	0.544	2.51
PickupBlend forecast / planned cap.	0.515	0.870	0.314	0.859	0.448	3.38
Decision tree	0.471	0.829	0.447	0.701	0.546	2.25
Gaussian naive Bayes	0.293	0.789	0.297	0.919	0.443	3.58

6.5 Per-fold view and bias

Table 13 separates the two folds. Every model under-forecasts on fold 4, the new season after the winter gap. The bias is 4 to 8 guests for the competitive models and up to 18 for the historical baselines. On fold 5, which continues a season already partly seen, biases are small or positive. Ridge extrapolates the season level linearly and has the smallest fold-4 bias. The networks, which do not see the season index, under-forecast the new level more but still have the lowest MAE on both folds.

Table 13: MAE and bias per fold.

Model	Fold 4 (new season)		Fold 5	
	MAE	Bias	MAE	Bias
MLP, greedy pick, 5-seed ensemble	24.14	−6.04	20.24	−0.65
MLP, greedy pick (3 seeds)	24.23	−5.44	20.35	−0.82
Ridge + interactions	25.89	−3.77	24.08	+3.39
PickupBlend	27.58	−7.89	24.34	+0.81
PickupRatio	32.03	−0.95	30.38	+6.45
SlotMonthDowMean	33.02	−14.83	27.79	−4.84
SeasonalNaive(14)	34.41	−17.37	33.66	+6.10
GlobalMean	50.35	−18.06	44.68	−0.15

What to expect on the test season. The 2026 test window has the structure of fold 4: a new season, forecast across a winter gap. The MLP does not see the season index, so it can only learn the new level from the book and the history features, and on fold 4 it fell short of it. Before opening the test we therefore declared the expectation of a test MAE near 24 for the final MLP, above the folds 4–5 mean, and a negative bias. The MAE was lower (21.74) and the bias more negative (−10.17).

6.6 Test season 2026

Table 14 gives the regression results on the 2026 window, scored once under the protocol of §5.8.

Table 14: Regression on the 2026 test season, ranked by MAE (guests per route-day). Brackets are 95% day-block bootstrap intervals. Day MAE is in guests per day; Day MAPE is the mean absolute percentage error of daily totals. Δ MAE is the difference to the final MLP, a 5-seed ensemble.

Model	MAE		R^2	Bias	Day	Day-	Day	Δ MAE
	[95% CI]	RMSE			MAE	MAE%	MAPE%	
MLP, final (primary)	21.74 [20.48, 23.03]	35.87	0.693	-10.17	555	24.6	26.5	-
Ridge + interactions	24.39 [23.24, 25.59]	35.88	0.693	-3.75	455	20.1	24.5 +2.66 [+1.73, +3.51]	
PickupBlend	27.43 [26.08, 28.77]	41.46	0.590	-7.47	540	23.9	27.8 +5.69 [+4.62, +6.75]	
SlotMonthDowMean	31.18	43.79	0.543	-13.49	664	29.4	31.5	+9.44
PickupRatio	34.48	57.02	0.225	-1.46	520	23.0	28.7	+12.74
SeasonalNaive(14)	34.58	48.02	0.450	-18.37	809	35.7	36.4	+12.85
GlobalMean	53.70	67.79	-0.095	-19.99	843	37.3	37.3	+31.96

The primary model. The final MLP reaches an MAE of 21.74 on the test season. This is below the declared expectation of about 24 and below its own folds 4–5 mean (22.19). It beats PickupBlend by 5.69 guests per departure (21%) and Ridge with interactions by 2.66. Both intervals lie well away from zero. The models rank in the same order as on the development folds.

Growth and bias. Mean guests per route-day were 34.8 in 2023, 50.3 in 2024 and 53.8 in 2025, and 66.5 in the 2026 test window, 24% more than in 2025. Every model under-forecasts. The MLP’s mean error is -10.17 , but its median error is only -3.38 . Part of the negative mean comes from the L1 loss: it targets the median, and demand is right-skewed. The rest comes from the jump in level, which the networks cannot see (§5.5).

Daily totals. At day level Ridge is the best model, with a day MAE of 455 guests (20.1%) and a bias of -127 guests per day. The MLP reaches 555 (24.6%), with a bias of -346 , and under-forecasts 76% of the days. Its per-departure bias of -10 adds up over the 34 departures of a day. PickupBlend reaches 540, with a bias of -254 . The MLP is therefore the better model for single departures, and Ridge is the better model for daily totals, for example for rostering.

Errors by month and segment. Table 15 breaks down the final MLP’s error. It beats PickupBlend in every month and in every segment. July and September have the largest negative bias. On the 136 holiday route-days the MLP is almost unbiased ($+1.94$), while it under-forecasts ordinary days by 10.55.

Worst days. Nine of the ten days with the largest error in the daily total are under-forecasts of 1,100–1,600 guests. Examples are Whit Sunday 2026-05-24 (3,363 guests against 1,765 forecast), Tuesday 2026-09-15 (2,923 against 1,386) and the Thursday and Friday after Whitsun, 2026-05-28 and 29 (2,370 against 1,183; 2,241 against 977). The tenth is an over-forecast: Friday 2026-08-21 (889 against 2,250). On these days demand differs from what the book and the calendar suggest. Weather is the most plausible cause, but we have not tested it (§7).

Fullness classification. Table 16 gives the classifiers on the test season. The prevalence is 0.179, higher than on the development folds (0.12–0.16), because 2026 is busier.

The primary classifier, the linear SVM, catches 736 of the 816 nearly-full departures (90%, against a target of 80%) with 646 false alarms, or 1.88 flagged departures per catch. Recall overshoots the target because the threshold was set on autumn 2025 days and 2026 is busier,

Table 15: Final MLP on the 2026 test season by month and segment, with PickupBlend’s MAE for reference.

Segment	Route-days	Mean guests	MLP MAE	MLP bias	PickupBlend MAE
May	612	52.2	21.57	−11.01	25.77
June	1,020	62.9	18.49	−6.83	24.63
July	1,054	75.7	23.96	−15.52	29.79
August	1,054	70.4	22.35	−4.01	30.44
September	816	65.1	22.25	−14.79	25.21
Empty T-7 book	1,039	12.6	9.11	−4.60	13.99
Non-empty T-7 book	3,517	82.5	25.47	−11.82	31.40
Holiday	136	74.8	22.65	+1.94	31.67
Ordinary day	4,420	66.3	21.71	−10.55	27.30

Table 16: Nearly-full classification on the 2026 test season. Thresholds were set for 80% recall on the last 15% of the development days. No-skill AP equals the prevalence, 0.179.

Model	AP	ROC-AUC	Precision	Recall	F1	Alarms/catch
Logistic regression	0.747	0.936	0.584	0.866	0.698	1.71
Linear SVM (primary)	0.738	0.937	0.533	0.902	0.670	1.88
MLP (final) forecast / planned cap.	0.730	0.924	0.518	0.881	0.652	1.93
PickupBlend forecast / planned cap.	0.571	0.882	0.434	0.884	0.582	2.31

so more departures score above it; precision drops accordingly. May is the weak month (35 of 62 caught), in line with the early-season under-forecast. From June to September the classifier caught 87–100%. Logistic regression and the linear SVM differ by 0.009 in AP, which we do not read as a difference. Both beat the PickupBlend flag clearly.

7 Limitations and next steps

Limitations.

- Because of the 2023 product launch, season-level growth is identified from 2024→2025 alone. Every model under-forecasts a new season, on fold 4 and on the test season (§6.6). No current feature gives the networks the next season’s level. A possible leading indicator is the individual-advance book: at T-7 it grew by 63% from 2024 to 2025, while final individual demand grew by 21%.
- The validation scores are optimistic. We selected configurations on folds 4–5, which gives a winner’s curse, and fold 4 covers only three months. The test season is unbiased, but it is a single season of 134 days.
- The comparison of greedy and random search rests on one seeded random sample of 21 configurations.
- We compared the four curve encoders at one fixed size (64 units, one layer), so their ranking holds at that size.
- Without cancellation timestamps, cancellations made before T-7 count as live, which slightly overstates the book (§2.1).
- We assume the planned seat split is fixed before the season and not edited in response to sales. The extract has no modification date to check this.
- Holiday effects are large, but each holiday occurs only about twice per training fold, too

rarely to estimate its effect. The holiday features have not produced a measurable gain.

- Weather is not included [D15]. The day-level residuals and the ten worst test days point to a daily driver such as weather. At T-7 only a forecast exists, so training must use archived forecasts; observed weather would give the model different inputs in training and in production. In mountain terrain, the choice of grid point and elevation (summit or valley) is a modelling question of its own.

Next steps.

1. Add archived weather forecasts as inputs and check whether they explain the worst test days.
2. Correct the systematic under-forecast of growth, for example with a level correction of the network's output or a trend input that the networks can use without extrapolating.
3. Choose the model by decision: the MLP for single departures, Ridge for daily totals such as rostering. Alternatively, reconcile the route-day forecasts to a day-level forecast.

A Decision log

Each entry records a decision, the alternatives we considered, the evidence that decided it and the choice. Under each decision are the sections that discuss it. Numbers are MAE on folds 4–5 unless stated otherwise.

ID	Decision	Alternatives	Evidence	Choice / consequence
Framing and data				
D1	Task, grain, horizon §1	Day grain; other lead times	Day grain $\approx$ 650 rows; route-days sum to days; T-7 is the last point to act	Route-day forecast at T-7; fullness flag as second task
D2	Seasons, population §3	2021–2026; drop zero-demand rows	2021–22: individual seats exactly 0, capacity 80 vs 188; $\approx$ 98% of no-booking route-days have target 0	2023–2026; drop 536 special + 2,041 disabled; keep zeros (16.3%); 22,558 rows
D3	Target §3	Bookings-derived total; stored counter	Horizon-0 reconciliation: corr 0.9999, 99.6% within ± 5	Stored DailyRoutes counter
D4	Test set §5.8, §6.6	Random holdout; last season	Test must be a future season	2026 sealed; end date pinned (2026-09-24); candidates and expectation declared first (final MLP 5-seed ens. primary; Ridge + interactions and five baselines as references; linear SVM primary classifier); fitted once on development set, scored once; day-block bootstrap CIs; 2026 aggregates seen in EDA never used for decisions
D5	Validation §5.1	Holdout; plain k -fold; expanding window	k -fold trains on the future, splits correlated days; equal blocks straddled winter; fold 2 dominated by the launch break	Expanding window, $k=5$, whole days, windows within a season; rank on folds 4–5, report per fold
T-7 snapshot				
D6	Cutoff rule §2.1, §3.5	none	Independent recompute; horizon-0 reconciliation	Leg counts only if created $\leq$ travel –7; history lags 14/21/28 (lag 7 falls on the cutoff day)
D7	Later-cancelled bookings §2.1	Include; exclude	Excluding: corr 0.6468 vs 0.6354, but uses post-T-7 information (14.3% of the book)	Include (no leak); pre-T-7 cancellations slightly overstate the book
D8	Streams §3.4	Raw booking types; behavioural streams	Visible at T-7 (dev): group 86.1%, advance 23.0%, on-spot 1.6%	Three streams; legacy nulls and unknown types $\rightarrow$ on-spot
D9	Departure identity §2.3	Train number; timetable slot	0 of 34 train numbers carry over 2024 $\rightarrow$ 2025; slot means corr 0.93–0.99	Slot (station + departure time)
D10	Empty-book staleness §3.5	Fill 60; sentinel + flag	Fill 60 made 17% of live books look staler than empty ones	Sentinel 999 + book_is_empty
Features				

continued on next page

ID	Decision	Alternatives	Evidence	Choice / consequence
D11	Per-day capacity columns §2.2	Keep; drop group only; drop group + total; drop all	Group = booked group seats in 95.5% (corr 0.9926); rebuilt exactly from total; then as 188 – (ind. + res.) on ≈80%: group MAE 1.96 vs 5.44, spurious ≈1.4 MAE gain	Drop all per-day allocation columns
D12	Planned capacity §3.7, §6.1	None; timetable split	Fixed before the season; Ridge + interactions 27.67 → 24.98, bias +7.8 → –0.2; actual total overridden on ≈15% (corr +0.29 with demand)	Planned group, individual, reserve and total seats from TimetableRoutes ; also the denominator of regression-derived flags
D13	Structural break (2023-10-27) §2.3, §3.7	As-is; step flag; down-weight; drop 2023	Season index reads the launch jump as growth (+60% individual 2023→24 vs +21% 2024→25, same calendar days)	Step dummy, keep all rows; growth identified from one transition
D14	Holidays §3.7	Binary flag; identity; federal / foreign / cantonal calendar	Sign reverses (+111% to +211% Whit Monday, –54% to –59% National Day); Central Swiss +38%; foreign no signal	Central Swiss cantonal calendar, identity encoded; little measurable gain
D15	Weather §7	Observed; archived forecasts	Only a forecast exists at T-7	Deferred; archived forecasts when added
Evaluation and linear models				
D16	Metrics §5.2	MSE only; per-row MAPE	MAPE undefined at zero demand; bias matters operationally	MAE headline; RMSE, R^2 , bias, day-level MAE%
D17	Bar to beat §4	Naive mean; planner heuristics	PickupBlend 25.96 vs GlobalMean 47.51	PickupBlend
D18	Preprocessing §5.3	Global scaling; integer codes	Codes are nominal; holiday effect changes sign	Per-fold pipeline: impute, standardise, one-hot; shared by all models
D19	Linear model §5.4, §6.1	OLS; Ridge; full degree 2; interactions; log target	OLS 26.41, Ridge 25.99; interactions 24.98 vs full degree 2 25.96 (season ² overshoots); log target 30.43, bias –7.9	Ridge + interactions on 10 drivers, $\alpha=1000$, raw target
Neural models				
D20	Early stopping §5.5	Stop on validation fold; inner split	Validation-fold stopping tunes on evaluation data	Chronological inner split: last 15% of training-fold days
D21	Season index §5.5	Give to networks; withhold	Unseen value at prediction; given: greedy pick 22.29 → 23.28, random pick 22.94 → 23.64; training on Ridge residuals did not help	Withheld from networks; Ridge keeps it
D22	Tuning protocol §5.5, §6.2	Single-seed one-factor search; greedy; random; Bayesian; Hyperband	Seed noise exceeds most setting differences; 21 configurations each: greedy 22.29, random 22.94	3-seed means, one-SE rule; random search (fewest parameters) and greedy (preference order); greedy wins
D23	Final MLP §6.2, §6.6	Greedy pick; random pick; single model or ensemble	Greedy 22.29 (5-seed ens. 22.19); random 22.94 (22.75); default 23.61	Greedy pick: [128,64], layer norm, dropout 0.3, L1, lr $3 \cdot 10^{-3}$, wd 10^{-4} , batch 4096; 5-seed ensemble; test 21.74 (PickupBlend 27.43)

continued on next page

ID	Decision	Alternatives	Evidence	Choice / consequence
D24	Booking-curve models §5.6, §6.3	Summary features only; CNN, LSTM, GRU or Transformer on the curve; with or without reference curve	Own curve: Transformer 25.07, GRU 25.07, CNN 25.14, LSTM 25.21 (tied), bias -12 ; + reference: Transformer 24.16, GRU 24.21, bias -3.3	Curve-only encoders at equal size (64 units, 1 layer); reference kept for the two best; not taken to the test (final MLP 22.19)
D25	Fullness classification §5.7, §6.4	F1-optimal threshold; target recall; threshold on validation fold	F1 thresholds depend on prevalence (fold 4 training tail 3.9% vs validation 16.4%)	Rank by AP; threshold for 80% recall and hyperparameter chosen on last 15% of training days; best: linear SVM, AP 0.650; test AP 0.738, recall 0.90, 1.88 alarms per catch